

Ablation

Component	1×		3×		10×	
	with	without	with	without	with	without
Full system , macro exact match (%)	85.0		85.0		86.7	
model calls per item (mean / cap)	1.0 / 1		3.0 / 3		4.4 / 10	
how a misread line gets fixed	code repairs only		outvoted by two other reads		code flags it, model re-reads it	
One read of the notes						
Code solver (<i>vs Granite answering directly</i>)	85.0	1.7	85.0	–	86.7	–
Worked examples in the prompt	85.0	51.7	85.0	–	86.7	–
Restate each line in plain words first	85.0	61.7	85.0	–	86.7	–
Code repair (no model calls)						
Wrong-name repair	85.0	78.3	85.0	80.0	86.7	80.0
Extra calls, 3×: read three times and vote						
2nd and 3rd read + per-line vote (<i>vs one read</i>)	–	–	85.0	85.0	86.7	85.0
Reads use different prompts (<i>vs one prompt</i> × 3)	–	–	85.0	80.0	86.7	83.3
Extra calls, 10×: re-ask only the lines code flags						
All targeted re-asks (<i>vs the 3-read vote alone</i>)	–	–	–	–	86.7	85.0
<i>each kind of re-ask removed on its own:</i>						
name check – <i>value not in its line</i>	–	–	–	–	86.7	85.0
missed line – <i>“no fact” but names people</i>	–	–	–	–	86.7	86.7
order – <i>who comes first</i>	–	–	–	–	86.7	86.7
line type – <i>kind of statement</i>	–	–	–	–	86.7	86.7
contradiction – <i>lines cannot both hold</i>	–	–	–	–	86.7	86.7

Table 1: The ablation. Macro exact match (%), mean of 1 runs on 60 items. Each row removes one component from the system at that budget and keeps everything else; “with” is always the full system at that budget. –: the component does not exist at that budget or was not run in this suite (see `--suite full`). “2nd and 3rd read” at 3× without = the 1× system.

How to read it. Read across a row to see what one component is worth at each budget: the gap between *with* and *without*. Read down the shaded rows for the cost–accuracy curve. The budgets are nested: 3× is the 1× system plus two more reads, and 10× is the 3× system plus targeted re-asks, so every 1× component is also inside 3× and 10×, and every 3× component inside 10×. With 60 items, one item moves the macro score by 1.7 points; differences smaller than the run-to-run spread ($\pm$) are noise.

How the extra calls fix errors, and why 3× and 10× do it differently. At 1× a misread line can only be caught by code: the wrong-name repair swaps or drops a fact that names someone its line never mentions. Nothing can recover a line the model missed. At 3× the system never asks the model to correct itself. It reads the whole note three times, each with a differently formatted prompt, and keeps, line by line, the fact at least two reads agree on. This fixes *random* misreads; it cannot fix a mistake all three reads share, which is why the reads use different prompts. At 10× code decides *which* lines need another look and *why*. After the vote it flags lines that break a check or on which the reads disagreed, groups them by kind of problem, and makes one call per kind, showing the model only those lines, the earlier readings, and what is wrong. A re-read replaces the vote only when code had found a concrete problem and the new answer passes the checks; otherwise it counts as one more vote. So 3× fixes errors by *agreement*, and 10× by *targeted correction*.

1 runs per system, 60 items. Command: `python3 ablate.py --suite main --runs 1 --workers 16`. Started 2026-10-01 22:55, took 12.6 min.

Supporting tables

Code repairs at $1\times$ (same model output)

	Unique	Ambiguous	Inconsistent	Macro	Calls/item	Lines right	Missed	Wrong
None	80.0	70.0	85.0	78.3	1.0	95.0	31	26
★ Wrong-name repair	85.0	75.0	95.0	85.0	1.0	95.5	35	16
Station/time swap repair	80.0	70.0	85.0	78.3	1.0	95.0	31	26
Both	85.0	75.0	95.0	85.0	1.0	95.5	35	16

Table 2: All four rows score the *same* $1\times$ model output, so differences come from the repair alone, not sampling. Wrong-name repair: a fact naming someone its own line never mentions is swapped to the one unused name the line does mention, else dropped. Swap repair: a station written where a time belongs (or the reverse) becomes the matching fact; it fires too rarely to matter and is not in the submitted system.

Where the $10\times$ calls go

	Items asked	Lines re-asked	Macro without it	Gain
3-read vote (the $3\times$ system)	–	–	85.0	–
+ name check (<i>a name, time or station the line does not mention</i>)	40%	37	85.0	+1.7
+ missed line (<i>read as no fact, but the restatement names people</i>)	33%	27	86.7	+0.0
+ order (<i>reads disagree on who comes first</i>)	17%	13	86.7	+0.0
+ line type (<i>reads disagree on the kind of statement</i>)	50%	47	86.7	+0.0
+ contradiction (<i>two lines cannot both hold</i>)	3%	5	86.7	+0.0
★ Full $10\times$	–	–	86.7	–

Table 3: After the 3-read vote, code flags lines and makes one call per kind of problem, only for items that have such lines. *Items asked*: share of items that got this call; *Lines re-asked*: lines sent, per run; *Gain*: full $10\times$ minus the system without this kind of call.

Declared versus true kind

True kind $\rightarrow$	$1\times$ U	A	I	$3\times$ U	A	I	$10\times$ U	A	I
Declared unique	17	0	0	18	0	1	18	0	1
Declared ambiguous	1	18	1	1	18	1	1	17	1
Declared inconsistent	2	2	19	1	2	18	1	3	18

Table 4: Mean item counts over runs for the submitted system at each budget (U = unique, A = ambiguous, I = inconsistent); the diagonal is correct. A missed constraint leaves too many solutions (a unique item declared ambiguous); a misread one usually makes the notes contradict (declared inconsistent).